

User Prompt: [excerpt from paper on where data was gathered]

I need to collect this data Copernicus

is the European Union's Earth Observation Programme, offering information services based on satellite Earth Observation and in-situ data [6]. The Copernicus Programme provides multiple services, including the Copernicus Marine Environment Monitoring Service (CMEMS), which delivers regular and systematic reference information on the physical and biogeochemical state, variability and dynamics of the ocean and marine ecosystems for the global ocean and the European regional seas [84]. In particular, this service provides data on SST and chl-a concentration along the Portuguese coast, which are freely and openly accessible to its users on the CMEMS website³ The SST measurements were obtained from the SST_GLO4 product. This satellite observation product is produced by the Operational SST and Ice Analysis (OSTIA) system and provides daily measurements of SST from January 2007 to the present day, on a global regular grid at 0.05° resolution. In its turn, the OCEANCOLOUR_ATL_CHL5 product was used to obtain the chl-a concentration measurements. This product is also a satellite-observation product that provides daily chl-a concentration measurements from September 1997 to the present day, on a regular longitude/latitude grid, from 20° N to 66° N in latitude and from 46° W to 13° E in longitude, with a resolution of 1 km. The data acquisition process was identical for SST and chl-a, and it will be described only for the latter. First, a subset operation was performed over the available data in the OCEANCOLOUR_ATL_CHL product to select only the chl-a concentration variable measured from January 2015 to December 2020. The data were provided on a netCDF format, which contained, for each time step and latitude/longitude grid point, a value of chl-a concentration. The coordinates for the sampling point in the RIAV1 production area were then used to choose the closest grid point, and a daily time series of chl-a concentration was obtained for that point. This time series was then merged with the toxins data by obtaining a weekly mean and joining the variables by the week of the year. Figure 3.3 shows the locations for the biotoxins sampling point in the RIAV1 area, the Copernicus products' satellite points and the IPMA meteorological station closest to this production area

ChatGPT Said:

Perfect — let's turn that paragraph into a **clear, practical step-by-step data acquisition workflow** so you can actually collect the data from **Copernicus Programme** via **Copernicus Marine Environment Monitoring Service (CMEMS)**.

You need:

- **SST** → SST_GLO4
- **Chlorophyll-a** → OCEANCOLOUR_ATL_CHL

- Location → RIAV1 production area (Portuguese coast)
 - Time range → Jan 2015 – Dec 2020
 - Final output → weekly averages aligned with toxin data
-

✅ STEP 1 — Create CMEMS Account

Go to the CMEMS website and:

1. Register for a free account
2. Accept licence
3. Generate API credentials (needed for automated download)

You will need:

username
password

✅ STEP 2 — Install Copernicus Marine Toolbox

Install the official client:

```
pip install copernicusmarine
```

(or use conda if preferred)

✅ STEP 3 — Identify Correct Dataset IDs

⚠ Important: Product names in papers are often legacy names.

You must search CMEMS catalog for current dataset IDs corresponding to:

 **SST product (OSTIA system)**

Originally called:

SST_GL04

Produced by:

Operational SST and Ice Analysis (OSTIA)

Likely current dataset resembles:

SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001

(but confirm in CMEMS catalog)

Chlorophyll-a product

Originally:

OCEANCOLOUR_ATL_CHL

Likely current dataset resembles:

OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_091

Confirm exact dataset ID in CMEMS portal.

STEP 4 — Subset the Data (Python Example)

Here is the modern API approach.

Example: Download Chlorophyll-a (2015–2020)

```
from copernicusmarine import subset
```

```
subset(
```

```
dataset_id="OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_091",
```

```
variables=["CHL"],
start_datetime="2015-01-01",
end_datetime="2020-12-31",
minimum_longitude=-9.5,
maximum_longitude=-9.5,
minimum_latitude=40.5,
maximum_latitude=40.5,
output_filename="chl_subset.nc"
)
```

Replace lat/lon with RIAV1 coordinates.

Example: Download SST

```
subset(
  dataset_id="SST_GLO_SST_L4_REP_OBSERVATIONS_010_011",
  variables=["analysed_sst"],
  start_datetime="2015-01-01",
  end_datetime="2020-12-31",
  minimum_longitude=-9.5,
  maximum_longitude=-9.5,
  minimum_latitude=40.5,
  maximum_latitude=40.5,
  output_filename="sst_subset.nc"
)
```

STEP 5 — Extract Closest Grid Point

Once downloaded (NetCDF format):

```
import xarray as xr

ds = xr.open_dataset("chl_subset.nc")

print(ds)
```

Select nearest grid point:

```
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)

chl_series = point["CHL"].to_pandas()
```

✅ STEP 6 — Convert Daily → Weekly Mean

```
chl_weekly = chl_series.resample("W").mean()
```

Do same for SST.

✅ STEP 7 — Merge With Toxins

Assuming toxins dataframe has weekly index:

```
merged = toxins_df.join(chl_weekly, how="inner")
```

Or explicitly:

```
merged = pd.merge(
    toxins_df,
    chl_weekly,
    left_on="week",
    right_index=True
)
```

⚠️ Important Technical Notes

Resolution differences:

- SST → 0.05° (~5km)

- CHL → 1km grid

That means nearest grid point selection is correct approach.

Key Concept

The paper describes:

1. Subset spatially
2. Extract nearest pixel
3. Convert daily → weekly
4. Merge by week-of-year

This is straightforward and reproducible.

What I Need From You

To give you a fully exact working script:

1. RIAV1 latitude?
 2. RIAV1 longitude?
 3. Do you want:
 - Manual website download?
 - Or fully automated Python script?
-

If you're planning to reproduce this for publication, I can also:

- Verify current CMEMS dataset IDs

- Ensure you're using reprocessed (REP) not near-real-time (NRT)
- Help you write a reproducible Methods section

Just tell me your goal (thesis replication vs new study).

User Prompt:

so i need to have an account or i cant get the data?

ChatGPT said:

Yes — you **do need a free account** to download data from the **Copernicus Marine Environment Monitoring Service (CMEMS)** portal.

But:

- ✓ It is completely free
- ✓ It is open-access
- ✓ No institutional subscription required
- ✓ No payment required

You just need to register and accept the licence agreement.

Why an account is required

The **Copernicus Programme** operates under an open data policy, but they require:

- User registration (for download tracking)
- Acceptance of licence terms
- API authentication for automated downloads

It's administrative — not paywalled.

What you can do without an account

You can:

- Browse the catalog
- View product descriptions
- See metadata

You cannot:

- Download NetCDF files
 - Use the API
 - Access bulk data
-

How long does it take?

Registration usually takes:

- 2–3 minutes
- Account activation is immediate

You'll get:

- Username
 - Password
 - API credentials
-

Important for Research Reproducibility

If you are publishing or doing a thesis, using CMEMS with an account is actually **the correct and expected workflow**, because:

- The datasets are versioned
 - Products are traceable
 - Licensing is clear
-

If you'd like, I can walk you through:

1. The exact dataset names you should select (current catalog versions)
2. Whether you should use **REP (reprocessed)** or **NRT (near-real-time)** products
3. A fully automated script once you have credentials

Just tell me your goal (replication vs new analysis).

User Prompt:

explain from what i sent you how the data was acquired

ChatGPT said:

Based strictly on what you sent, here is **exactly how the data were acquired and processed**, step by step.

1 Data Source

All environmental data came from the **Copernicus Marine Environment Monitoring Service (CMEMS)**, which is part of the **Copernicus Programme**.

CMEMS provides satellite-derived ocean variables, including:

- Sea Surface Temperature (SST)
- Chlorophyll-a concentration (chl-a)

These datasets are freely accessible through the CMEMS website.

2 Products Used

Two specific satellite products were used:

Sea Surface Temperature (SST)

Product:

SST_GLO4

- Produced by the Operational SST and Ice Analysis (OSTIA) system
- Daily measurements
- Global coverage
- 0.05° spatial resolution (~5 km)
- Available from January 2007 onward

Chlorophyll-a (chl-a)

Product:

OCEANCOLOUR_ATL_CHL

- Daily chlorophyll-a concentration
- Spatial coverage:
 - Latitude: 20° N to 66° N
 - Longitude: 46° W to 13° E
- 1 km spatial resolution
- Available from September 1997 onward

3 Temporal Subsetting

A subset operation was performed:

- Variable selected: **chlorophyll-a concentration**
- Time range selected: **January 2015 to December 2020**

Although only chl-a is described in detail, the text states that:

“The data acquisition process was identical for SST and chl-a.”

So the same temporal filtering was applied to SST.

4 File Format

The data were downloaded in **NetCDF format**.

Each file contained:

- A time dimension (daily time steps)
- A latitude grid
- A longitude grid
- A value of chl-a (or SST) at each grid point and time step

So the raw data structure was:

`time × latitude × longitude`

5 Spatial Extraction (RIAV1 Point)

The coordinates of the RIAV1 production area sampling point were used.

From the gridded dataset:

- The **closest satellite grid point** to the RIAV1 coordinates was selected.
- A **daily time series** was extracted from that grid point.

So they did NOT average spatially.
They selected the nearest pixel.

6 Temporal Aggregation

The daily time series was converted to:

A **weekly mean**

This means:

- Daily chl-a → averaged per week
 - Daily SST → averaged per week
-

7 Merging With Toxin Data

The weekly environmental time series was merged with toxin data by:

Joining variables by the **week of the year**

So the final dataset had:

| Week | Toxins | SST | chl-a |

All aligned weekly.

8 Summary of Workflow

In plain workflow form:

1. Access CMEMS products (SST_GLO4 and OCEANCOLOUR_ATL_CHL)
2. Subset time range (2015–2020)
3. Download NetCDF files
4. Select nearest grid point to RIAV1

5. Extract daily time series
 6. Convert daily → weekly mean
 7. Merge with toxin dataset by week
-

Key Technical Characteristics

- Satellite-derived data
 - No in-situ measurements used here
 - Nearest-neighbour spatial selection
 - Weekly temporal aggregation
 - Grid resolutions:
 - SST → 0.05°
 - chl-a → 1 km
-

If you want, I can now:

- Translate this into a clean Methods section paragraph for a thesis
- Or turn it into a reproducible Python workflow
- Or identify potential sources of bias in this acquisition method

User Prompt:

so what i need to do is collect the data from 2021 to 2024 for these two things

ChatGPT said:

Yes — exactly.

To extend the dataset, you need to collect **the same two variables**, using the same procedure, but for:

January 2021 – December 2024

And you must keep everything consistent with the original workflow.

What You Need to Download

From the **Copernicus Marine Environment Monitoring Service (CMEMS)**:

1 Sea Surface Temperature (SST)

Originally referred to as:

SST_GL04 (OSTIA system)

You need:

- Daily SST
 - Same spatial resolution ($\sim 0.05^\circ$)
 - Same product family (preferably reprocessed if matching historical data)
 - Time range: 2021–2024
 - Same spatial point (nearest grid to RIAV1)
-

2 Chlorophyll-a (chl-a)

Originally referred to as:

OCEANCOLOUR_ATL_CHL

You need:

- Daily chl-a concentration

- Same 1 km resolution Atlantic product
 - Time range: 2021–2024
 - Same spatial point (nearest grid to RIAV1)
-

Very Important — Consistency

You must:

- Use the **same product version type** (REP vs NRT)
- Use the same spatial extraction method (nearest pixel)
- Use daily data
- Convert to weekly mean the same way
- Merge by week exactly as before

Otherwise you introduce structural differences into your time series.

REP vs NRT (Important Decision)

CMEMS usually provides:

- **REP (Reprocessed)** → Stable, quality-controlled, used for research
- **NRT (Near Real Time)** → Operational, updated daily

If your 2015–2020 data came from reprocessed datasets, you should:

Continue with REP for 2021–2024 if available

If REP is not available for recent years, you may need to use NRT — but document it.

Conceptually What You're Doing

You are not changing methodology.

You are simply extending:

2015–2020

→

2015–2024

By appending new environmental data.

Practical Steps

1. Log into CMEMS
2. Select same SST product
3. Select same chl-a product

Set date range to:

2021-01-01

2024-12-31

- 4.
 5. Subset spatially around RIAV1
 6. Download NetCDF
 7. Extract nearest grid point
 8. Convert daily → weekly mean
 9. Append to existing weekly dataset
-

Critical Check

Before merging, verify:

- Units are identical
 - Variable names match
 - No scaling differences
 - Time stamps align correctly
 - No missing-value format change
-

If you tell me:

- Your RIAV1 latitude & longitude
- Whether you are using Python or manual website download

I can give you an exact reproducible script for 2021–2024.

User Prompt:

I used these links from the paper but the links do not have the data i need
4https://resources.marine.copernicus.eu/?option=com_csw&view=details&product_id=ST_GLO_SST_L4_NRT_OBSERVATIONS_010_001
5https://resources.marine.copernicus.eu/?option=com_csw&view=details&product_id=OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_098

ChatGPT said:

You're right — simply opening the **old product pages you linked** doesn't give you the data you want because:

1. **The CMEMS catalogue has changed over time** — products get *renamed*, *replaced*, or *superseded* in the official data store.
2. **The static “details” pages** you found are metadata/description pages — not direct download links.

So your URLs (e.g., [SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001](https://resources.marine.copernicus.eu/?option=com_csw&view=details&product_id=ST_GLO_SST_L4_NRT_OBSERVATIONS_010_001)) are **product identifiers**, not direct data access links — and the actual files must still be retrieved from the *Copernicus Data Store* interface after you log in.

What These Product IDs Actually Are

✓ SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001

- This is a **near-real-time (NRT) SST product** produced by the CMEMS SST Thematic Assembly Center (TAC).
- It is *described* in the SST Product User Manual, but the page you saw isn't the data itself.
- The real download happens *inside* the Copernicus Data Store after logging in.

CMEMS organizes products with identifiers like:

SST_GLO-SST-L4-NRT-OBSERVATIONS-010-001

which you then use as the dataset identifier when downloading.

✓ OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_098

- This is a **Level-4 reprocessed Atlantic chlorophyll-a product**.
- Reprocessed products (REP) are typically long historical records (e.g., 1997–2018) and are *updated only periodically*.
- The page you found is metadata — not the raw data itself.

Why the Pages “Don’t Have the Data”

The links you tried are the descriptive metadata pages from the **Copernicus Marine Catalogue**. They:

- Describe the product
- List variables and resolution
- Show temporal and spatial coverage

...but **do not contain the actual NetCDF files** themselves.

To get data you must use the **DATA ACCESS** section of the Copernicus Marine Data Store.

What You Must Do Instead

✓ Step 1 — Register on Copernicus Marine Data Store

You *must* have a free account to access data (even if search and metadata are open to all).

✓ Step 2 — Search for the Correct Dataset in the Data Store

Use the product IDs you have as search terms in the catalogue:

- SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001
- OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_098

You will then see:

- Available time coverage
- Download options
- Subset configuration (time, lat/lon)

✓ Step 3 — Subset by Time and Space

Once logged in, use:

- The GUI subsetter
- Or the Copernicus Marine Toolbox API

to extract only the dates and geographical region you need.

Notes About Product Versions and Time Coverage

Because you want data **2021–2024**, pay attention to:

SST

- The **NRT product** should cover daily SST up to near present. But always confirm the actual *latest date available* in the Copernicus Data Store.

Chlorophyll-a

- The **REP Atlantic product (009_098)** *might not include data past some year* — because reprocessed products are often not updated frequently.
- For recent years you may need the **NRT version** (`OCEANCOLOUR_ATL_CHL_L4_NRT_OBSERVATIONS_009_0XX`) instead.

If REP stops short, use the NRT version *for your 2021–2024 range*, then document that choice.



How to Download Once Found

After locating the dataset in the Data Store, you can download:

Option A — GUI Subset

- Choose time range (e.g., 2021-01-01 → 2024-12-31)
- Choose bounding box around RIAV1
- Choose variables (SST or chl-a)
- Download NetCDF

Option B — Python Toolbox (automated)

You can use the Python *Copernicus Marine Toolbox* to download a subset by:

1. Dataset ID
2. Time range
3. Spatial bounding box

Example usage is typically of the form:

```
from copernicusmarine import subset

subset(
    dataset_id="...",
    variables=["..."],
    start_datetime="YYYY-MM-DD",
    end_datetime="YYYY-MM-DD",
    minimum_longitude=...,
    maximum_longitude=...,
    minimum_latitude=...,
    maximum_latitude=...,
    output_filename="..."
)
```

✓ Summary

- The **links you found are product descriptions — not the data itself**.
 - To get the actual data for 2021–2024 you must:
 - Register on the CMEMS Data Store
 - Search by product ID
 - Download or subset data in *netCDF* format
 - Extract nearest-grid values and aggregate weekly
-

If you want, I can now:

- ✓ Find the **current exact CMEMS dataset IDs** that contain data through 2024 for SST and chlorophyll-a
- AND
- ♦ Provide direct Python code to download them automatically once you have credentials

Just tell me whether you want **REP or NRT** for chl-a (or both) and I'll fetch the precise names and examples.

User Prompt: [snippet of description of data]

Full name Global Ocean OSTIA Sea Surface Temperature and Sea Ice Analysis
Product ID SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001 Source Satellite
observations Spatial extent Global OceanLat -89.97° to 89.97°Lon -179.98° to 179.98°
Spatial resolution 0.05° × 0.05° Temporal extent 16 Jan 2024 to 16 Feb 2026 Temporal
resolution Daily Processing level Level 4 Variables Sea ice area fractionSea surface
temperature Feature type Grid Blue markets Polar Environment MonitoringClimate &
AdaptationPolicy & GovernanceScience & InnovationExtremes & Hazards &
SafetyCoastal Services Coordinate reference system WGS 84 (EPSG:4326) Update
frequency Daily – 12:00 Format NetCDF-3NetCDF-4 Originating centre Met Office (UK)
Last metadata update 25 November 2025

ChatGPT said:

Good — this is actually very informative.

What you're looking at is:

Product ID:

SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001

Full name:

Global Ocean OSTIA Sea Surface Temperature and Sea Ice Analysis

The Important Part

Look at the **temporal extent**:

16 Jan 2024 to 16 Feb 2026

This means:

 This specific dataset only contains very recent Near-Real-Time (NRT) data.

It does **NOT** contain:

- 2021
- 2022
- 2023

- Most of 2024

That's why you aren't seeing the years you need.

Why This Is Happening

CMEMS splits SST products into:

1. **NRT (Near Real Time)** → recent operational data
2. **REP (Reprocessed)** → long historical archive

You opened the **NRT version**, which only keeps a rolling recent window.

For your 2021–2024 data, you need the **Reprocessed (REP)** dataset.

What You Actually Need

Instead of:

`SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001`

You need something like:

`SST_GLO_SST_L4_REP_OBSERVATIONS_010_011`

(or the current REP equivalent in the catalog)

The REP version contains the full historical record from 2007 onward.

What To Do Next

In the Copernicus Marine Data Store:

Search for:

SST_GLO_SST_L4_REP

- 1.
2. Select the **Reprocessed (REP)** product.

Confirm temporal coverage includes:

2021-01-01 to 2024-12-31

- 3.
-

Same Logic for Chlorophyll-a

If the Atlantic chl-a REP product stops before 2024, then:

- Use REP up to the last available year
 - Use NRT to fill remaining years
 - Document this clearly in methods
-

Summary

The dataset you opened is correct in type (OSTIA SST L4), but:

-  It is the NRT version
 -  It does not include 2021–2023
 -  You need the REP version instead
-

If you want, tell me:

- Are you downloading manually via GUI?
- Or do you want the exact dataset IDs for both SST and chl-a that will cover 2021–2024?

I'll guide you precisely so you don't click the wrong one again.

User Prompt: [copy of data description]

Full name Global Ocean OSTIA Sea Surface Temperature and Sea Ice Reprocessed
Product ID SST_GLO_SST_L4_REP_OBSERVATIONS_010_011 Source Satellite
observations Spatial extent Global OceanLat -89.97° to 89.97°Lon -179.98° to 179.98°
Spatial resolution 0.05° × 0.05° Temporal extent 30 Sep 1981 to 30 Dec 2023 Temporal
resolution Daily Processing level Level 4 Variables Sea ice area fractionSea surface
temperature Feature type Grid Blue markets Polar Environment MonitoringClimate &
AdaptationPolicy & GovernanceScience & InnovationExtremes & Hazards &
SafetyCoastal Services Coordinate reference system WGS 84 (EPSG:4326) Update
frequency Notapplicable Format NetCDF-4 Originating centre Met Office (UK) Last
metadata update 25 November 2025

ChatGPT said:

Perfect — this is the **correct historical SST dataset**.

You have:

Product ID:

SST_GLO_SST_L4_REP_OBSERVATIONS_010_011

Temporal extent:

30 Sep 1981 → 30 Dec 2023

Important Conclusion

This dataset gives you:

- ✓ 2021
- ✓ 2022
- ✓ 2023

But **NOT 2024**.

So to build a continuous 2021–2024 SST series, you must:

Use **REP (010_011)** for:

2021-01-01 → 2023-12-30

-

Use **NRT (010_001)** for:

2024-01-01 → 2024-12-31

-

Then concatenate them.

Why This Split Exists

CMEMS standard workflow:

- **REP** = stable, quality-controlled archive (ends at last completed reprocessing year)
- **NRT** = operational stream (covers most recent ~2 years)

So your SST timeline should be:

1981 ----- 2023 | 2024 – present
 REP NRT

This is normal and expected.

What You Should Do (SST)

Download:

1 REP

SST_GLO_SST_L4_REP_OBSERVATIONS_010_011

Time range:

2021-01-01 → 2023-12-30

2 NRT

SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001

Time range:

2024-01-01 → 2024-12-31

Then:

- Extract nearest RIAV1 grid point
 - Convert daily → weekly mean
 - Concatenate
 - Merge with toxins
-

Is Mixing REP and NRT Allowed?

Yes — scientifically this is standard practice.

But in your Methods section you must state something like:

SST data from 2021–2023 were obtained from the reprocessed OSTIA Level-4 product (CMEMS product ID 010_011), while 2024 data were obtained from the near-real-time version (010_001).

Important Check Before Merging

Before concatenating REP and NRT:

Confirm:

- Same variable name (likely `analysed_sst`)
- Same units (Kelvin vs Celsius — OSTIA is usually Kelvin)
- Same spatial resolution (0.05° — confirmed)
- No structural differences in NetCDF dimensions

🎯 Now We Must Check Chlorophyll

Do the same check for:

OCEANCOLOUR_ATL_CHL_L4_REP_OBSERVATIONS_009_098

Paste its temporal extent like you did for SST, and I'll tell you if the same REP/NRT split is required.

You're very close to having the full extension correctly structured.

the data was downloaded as a .nc file what do i do with that

Perfect — .nc (NetCDF) is exactly what you should get. That's the standard scientific format for CMEMS data.

Now I'll walk you through **exactly what to do next**, step by step.

🧠 What a .nc File Contains

Your file contains a 3D grid:

time × latitude × longitude

For each day and each grid point, it stores:

- SST (likely analysed_sst)
or
- Chlorophyll-a (CHL or similar)

You now need to:

1. Open the file
2. Extract the grid point closest to RIAV1
3. Convert daily → weekly mean

4. Save it for merging

✅ Step 1 — Install Required Packages

If not installed:

```
pip install xarray netcdf4 pandas
```

✅ Step 2 — Open the NetCDF File

```
import xarray as xr

ds = xr.open_dataset("your_file.nc")

print(ds)
```

This will show:

- Variable names
 - Coordinates
 - Dimensions
-

✅ Step 3 — Identify Variable Name

For SST, likely:

```
ds.data_vars
```

You might see:

```
analysed_sst
```

For chlorophyll:

CHL

(Always check with `print(ds)`.)

✅ Step 4 — Extract Nearest Grid Point (RIAV1)

You need RIAV1 latitude & longitude.

Example:

```
RIAV1_LAT = 40.60  
RIAV1_LON = -8.73
```

Then:

```
point = ds.sel(  
    latitude=RIAV1_LAT,  
    longitude=RIAV1_LON,  
    method="nearest"  
)
```

✅ Step 5 — Convert to Pandas Time Series

For SST:

```
sst_series = point["analysed_sst"].to_pandas()
```

For chlorophyll:

```
chl_series = point["CHL"].to_pandas()
```

Now you have:

DatetimeIndex → daily values

⚠ Important — SST Units

OSTIA SST is usually in **Kelvin**.

If so, convert to Celsius:

```
sst_series = sst_series - 273.15
```

Check units:

```
ds["analysed_sst"].attrs
```

✅ Step 6 — Convert Daily → Weekly Mean

```
weekly_series = sst_series.resample("W").mean()
```

This creates weekly averages (Sunday-based by default).

If your toxin data uses ISO weeks, use:

```
weekly_series = sst_series.resample("W-MON").mean()
```

Match whatever your toxin dataset uses.

✅ Step 7 — Save to CSV (Optional)

```
weekly_series.to_csv("sst_weekly_2021_2024.csv")
```

✅ Step 8 — Concatenate REP and NRT (If Needed)

If you downloaded:

- REP 2021–2023

- NRT 2024

Do:

```
combined = pd.concat([weekly_rep, weekly_nrt])
combined = combined.sort_index()
```

Quick Sanity Checks

Before merging:

```
print(combined.head())
print(combined.tail())
print(combined.isna().sum())
```

Ensure:

- No large missing chunks
 - Values look realistic
 - No unit mismatch
-

Final Output Should Be

A clean weekly time series like:

Date	SS T
2021-01-03	14. 2
2021-01-10	13. 9
...	...

Then merge with toxins by date.

If you paste:

- The output of `print(ds)`
- Your RIAV1 coordinates

I can give you an exact tailored extraction snippet for your file.

User Prompt:

can you provide a script that does this

ChatGPT said:

Yes — here is a **clean, complete script** that:

1. Opens a CMEMS `.nc` file
2. Extracts the nearest grid point to RIAV1
3. Converts SST from Kelvin → Celsius (if needed)
4. Converts daily → weekly mean
5. Saves the result to CSV

You can use this for **SST or chlorophyll** (just change the variable name).

✅ Full Script — NetCDF → Weekly Time Series

Save this as:

`extract_weekly_timeseries.py`

```

#!/usr/bin/env python3
"""
Extract weekly time series from CMEMS NetCDF file
-----
1. Opens .nc file
2. Extracts nearest grid point
3. Converts daily to weekly mean
4. Saves CSV
"""

import xarray as xr
import pandas as pd
import numpy as np

# ===== CONFIG ===== #

NC_FILE = "your_file.nc"    # <-- change to your file path
VARIABLE_NAME = "analysed_sst" # e.g. "analysed_sst" or "CHL"

RIAV1_LAT = 40.60    # <-- replace with correct latitude
RIAV1_LON = -8.73    # <-- replace with correct longitude

OUTPUT_CSV = "weekly_output.csv"

# ===== #

print("Opening NetCDF file...")
ds = xr.open_dataset(NC_FILE)

print("\nDataset summary:")
print(ds)

# ----- Extract nearest grid point -----
print("\nExtracting nearest grid point...")
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)

```

```

# ----- Convert to pandas series -----
series = point[VARIABLE_NAME].to_pandas()

print("\nFirst few daily values:")
print(series.head())

# ----- Check units -----
attrs = ds[VARIABLE_NAME].attrs
print("\nVariable attributes:")
print(attrs)

# If SST and in Kelvin, convert to Celsius
if VARIABLE_NAME.lower().startswith("analysed_sst"):
    if "units" in attrs and "K" in attrs["units"]:
        print("Converting Kelvin to Celsius...")
        series = series - 273.15

# ----- Daily → Weekly -----
print("\nResampling to weekly mean...")
weekly = series.resample("W-MON").mean()

print("\nFirst few weekly values:")
print(weekly.head())

# ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV)

print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")

```

How To Use

1. Replace:

- "your_file.nc"
- VARIABLE_NAME

- RIAV1_LAT
- RIAV1_LON

2. Run:

```
python extract_weekly_timeseries.py
```

For Chlorophyll

Change:

```
VARIABLE_NAME = "CHL "
```

(No Kelvin conversion needed.)

If You Have REP and NRT Files

Run this script twice:

- Once for REP file
- Once for NRT file

Then combine:

```
rep = pd.read_csv("rep_weekly.csv", index_col=0,
parse_dates=True)
nrt = pd.read_csv("nrt_weekly.csv", index_col=0,
parse_dates=True)

combined = pd.concat([rep, nrt]).sort_index()
combined.to_csv("sst_2021_2024_weekly.csv")
```

Final Output

You will have:

```
Date, value
2021-01-04, 14.23
2021-01-11, 13.98
...
```

Ready to merge with toxins.

If you'd like, I can also provide:

- A script that automatically merges REP + NRT in one run
- A script that processes both SST and CHL together
- A script that directly merges with your toxins dataset

Just tell me which workflow you want.

User Prompt:

how do you know that is the proper lon and lat

ChatGPT said:

Excellent question — and you're right to challenge that.

I **do not know** that 40.60, -8.73 is correct.

I just used example coordinates.

You must use the **exact RIAV1 sampling coordinates**, otherwise you may extract the wrong ocean pixel.

✅ How To Get the Correct RIAV1 Coordinates

There are three proper ways:

1 From Your Toxins Dataset (Best Option)

Check your toxins data source (IPMA or monitoring agency).

Often the sampling station metadata includes:

- Latitude
- Longitude

Look in:

- The original raw data file
- A metadata PDF
- A station information table

If you share where your toxin data came from, I can help you locate it.

2 From the Paper (Figure 3.3)

The paper mentions:

“Figure 3.3 shows the locations for the biotoxins sampling point in the RIAV1 area...”

If the coordinates are listed in the text or supplementary material, use those.

Sometimes they are hidden in:

- Figure captions
 - Appendix tables
 - Data availability section
-

3 Use a Map (If Official Coordinates Not Provided)

If RIAV1 is a known aquaculture production area in Portugal, you can:

1. Open Google Maps
2. Search "RIAV1 Portugal"
3. Right-click the sampling point
4. Copy coordinates

But this should only be done if official metadata is unavailable.

Why Precision Matters

SST resolution:

$0.05^\circ \approx \sim 5 \text{ km}$

User Prompt: [copy of error message when executing code]

I am getting this error in this script

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE = "METOFFICE-GLO-SST-L4-REP-OBS-SST_1771427025069.nc" # <--
change to your file path VARIABLE_NAME = "analysed_sst" # e.g. "analysed_sst" or
"CHL" RIAV1_LAT = 40.675 # <-- replace with correct latitude RIAV1_LON = -8.73 # <--
replace with correct longitude OUTPUT_CSV = "weekly_output.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds) #
----- Extract nearest grid point ----- print("\nExtracting nearest grid
point...") point = ds.sel( latitude=RIAV1_LAT, longitude=RIAV1_LON, method="nearest"
) # ----- Convert to pandas series ----- series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) # ----- Check units ----- attrs =
ds[VARIABLE_NAME].attrs print("\nVariable attributes:") print(attrs) # If SST and in
Kelvin, convert to Celsius if VARIABLE_NAME.lower().startswith("analysed_sst"): if
"units" in attrs and "K" in attrs["units"]: print("Converting Kelvin to Celsius...") series =
series - 273.15 # ----- Daily → Weekly ----- print("\nResampling to
weekly mean...") weekly = series.resample("W-MON").mean() print("\nFirst few weekly
values:") print(weekly.head()) # ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")
```

Traceback (most recent call last): File

```
"/home/jorge/Chatgpt/HABS/sst_data/prepare_data.py", line 28, in <module> ds =  
xr.open_dataset(NC_FILE) File  
"/home/jorge/anaconda3/envs/HABS/lib/python3.13/site-packages/xarray/backends/api.  
py", line 588, in open_dataset engine = plugins.guess_engine(filename_or_obj) File  
"/home/jorge/anaconda3/envs/HABS/lib/python3.13/site-packages/xarray/backends/plugin.  
py", line 217, in guess_engine raise ValueError(error_msg) ValueError: found the  
following matches with the input file in xarray's IO backends: ['netcdf4', 'h5netcdf']. But  
their dependencies may not be installed, see:  
https://docs.xarray.dev/en/stable/user-guide/io.html  
https://docs.xarray.dev/en/stable/getting-started-guide/installing.html
```

ChatGPT said:

Good — this is a **dependency issue**, not a problem with your script.

The key part of the error is:

```
found the following matches ... ['netcdf4', 'h5netcdf']  
But their dependencies may not be installed
```

That means:

- `xarray` sees your file is a NetCDF file.
- It knows which engines can open it.
- But the required backend library is missing in your conda environment.

✓ What's Happening (Simple Explanation)

`xarray` does not read NetCDF files by itself.
It relies on one of these libraries:

- `netCDF4`
- `h5netcdf`

Neither is installed in your `HABS` environment.

✅ Fix (Recommended Solution)

Since you're using conda, run this:

```
conda activate HABS
conda install -c conda-forge netcdf4
```

That is usually the cleanest fix.

Alternative (if conda install fails)

```
pip install netcdf4
```

But with conda environments, prefer the conda-forge install.

Optional: Be Explicit in Your Script

After installing, you can make your script more robust by specifying the engine:

```
ds = xr.open_dataset(NC_FILE, engine="netcdf4")
```

This avoids ambiguity.

Quick Check After Installing

Run:

```
python -c "import netCDF4; print('netCDF4 installed correctly')"
```

If that works, your script should run.

Why This Happens

You created a fresh conda environment (HABS), and while it has:

- Python
- xarray
- pandas
- numpy

It likely does NOT automatically include NetCDF backends.

This is very common when working with CMEMS or satellite .nc files.

If you want, I can also help you verify that your latitude/longitude names match the file structure — sometimes CMEMS uses `lat` / `lon` instead of `latitude` / `longitude`, which would be the next potential issue.

User Prompt: [copy of code for cleaning SST data]

my data came back blank why is that

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE = "METOFFICE-GLO-SST-L4-NRT-OBS-SST-V2_1771510247814.nc" # <--
change to your file path VARIABLE_NAME = "analysed_sst" # e.g. "analysed_sst" or
"CHL" RIAV1_LAT = 40.675 # <-- replace with correct latitude RIAV1_LON = -8.73 # <--
replace with correct longitude OUTPUT_CSV = "weekly_output_NRT.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds) #
----- Extract nearest grid point ----- print("\nExtracting nearest grid
point...") point = ds.sel( latitude=RIAV1_LAT, longitude=RIAV1_LON, method="nearest"
) # ----- Convert to pandas series ----- series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) # ----- Check units ----- attrs =
ds[VARIABLE_NAME].attrs print("\nVariable attributes:") print(attrs) # If SST and in
Kelvin, convert to Celsius if VARIABLE_NAME.lower().startswith("analysed_sst"): if
"units" in attrs and "K" in attrs["units"]: print("Converting Kelvin to Celsius...") series =
series - 273.15 # ----- Daily → Weekly ----- print("\nResampling to
weekly mean...") weekly = series.resample("W-MON").mean() print("\nFirst few weekly
values:") print(weekly.head()) # ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")
```

ChatGPT said:

Good — this is very common with CMEMS files.

If your output CSV is blank (all NaNs or empty), one of these is almost certainly happening:

Most Likely Causes

❶ Wrong Coordinate Names

Many CMEMS NetCDF files use:

`lat / lon`

NOT

`latitude / longitude`

If those dimensions don't exist, `.sel()` can silently fail or return an empty structure.

Check this immediately:

Add:

```
print(ds.coords)
```

If you see:

```
lat
lon
time
```

Then change your selection to:

```
point = ds.sel(
    lat=RIAV1_LAT,
    lon=RIAV1_LON,
    method="nearest"
```

)

This is the most common issue.

2 Longitude Range Mismatch (VERY COMMON)

CMEMS global products often use:

0 to 360

for longitude — not -180 to 180.

You are using:

`RIAV1_LON = -8.73`

If the dataset uses 0–360, then:

`-8.73 ≠ 351.27`

So your point selection will land somewhere wrong or outside the expected region.

Check longitude range:

```
print(ds.longitude.min().values, ds.longitude.max().values)
```

or

```
print(ds.lon.min().values, ds.lon.max().values)
```

If you see something like:

`0.0 359.9`

Then convert your longitude:

```
RIAV1_LON = 360 - 8.73    # 351.27
```

This is extremely common with CMEMS SST products.

3 The Variable Is All Fill Values

Sometimes the grid cell selected is land or masked.

Check this:

```
print(point[VARIABLE_NAME].values)
```

If you see something like:

```
-32768
```

or

```
nan
```

Then the nearest point is invalid (possibly land).

Try printing the selected coordinates:

```
print(point)
```

4 Time Dimension Name Is Not "time"

CMEMS sometimes uses:

```
time
```

But occasionally:

```
analysis_time
```

If `.to_pandas()` returns empty, check:

```
print(ds.dims)
print(ds.coords)
```

✅ What I Suspect Most

Given you're using:

METOFFICE-GLO-SST-L4-NRT-OBS-SST-V2

That dataset typically uses:

- lat
- lon
- longitude from 0–360

So the most likely issue is longitude range.

🔍 Quick Debug Version

Insert this after opening dataset:

```
print("Coordinates:")
print(ds.coords)

print("Longitude range:")
print(ds.lon.min().values, ds.lon.max().values)

print("Latitude range:")
print(ds.lat.min().values, ds.lat.max().values)
```

Then adjust accordingly.

🎯 90% Probability Fix

Replace:

```
latitude=RIAV1_LAT,  
longitude=RIAV1_LON,
```

with:

```
lat=RIAV1_LAT,  
lon=360 - 8.73,
```

and try again.

If you paste the `print(ds)` output here, I can tell you the exact fix immediately.

User Prompt:

Dataset summary: <xarray.Dataset> Size: 20kB Dimensions: (time: 350, latitude: 4, longitude: 3) Coordinates: * time (time) datetime64[ns] 3kB 2024-01-17 2024-01-18 ... 2024-12-31 * latitude (latitude) float32 16B 40.08 40.12 40.17 40.22 * longitude (longitude) float32 12B -8.925 -8.875 -8.825 Data variables:

ChatGPT said:

Perfect — this output tells us exactly what's happening.

Your dataset contains:

- **Latitude range:** 40.08 → 40.22
- **Longitude range:** -8.925 → -8.825
- Only **4 × 3 grid cells**

But your requested coordinates are:

```
RIAV1_LAT = 40.675  
RIAV1_LON = -8.73
```

 40.675 is **outside** the dataset's latitude range.

Your dataset only covers:

40.08 – 40.22

That's why your extracted series is blank.

Even with `method="nearest"`, xarray cannot select something that is completely outside the available coordinate bounds.

✅ Why This Happened

You downloaded a **very small spatial subset** of the global SST product.

Your NetCDF file only contains a tiny box around ~40.1°N.

Your RIAV1 coordinate (40.675°N) is ~0.45° north of the available grid.

That's about 50 km away.

So there is no valid grid cell to select.

🔍 How to Fix It

Option 1 — Check if the latitude is wrong

Verify the true RIAV1 coordinate.

If RIAV1 is actually around 40.1°N, then update:

```
RIAV1_LAT = 40.17    # pick one from dataset
```

Option 2 — Re-download CMEMS data with larger spatial bounds

When downloading from CMEMS, expand the spatial bounding box so it includes:

Latitude: 40.6+

Longitude: -8.7+

Right now your file simply does not include that region.

Quick Test

Try this:

```
print("Dataset latitude values:", ds.latitude.values)
print("Dataset longitude values:", ds.longitude.values)
```

You'll see exactly the 4×3 grid you're limited to.

Important Concept

Your script is correct.

The issue is not:

- Not xarray
- Not resampling
- Not NetCDF
- Not Kelvin conversion

The issue is:

The requested point is outside the spatial extent of the NetCDF file.

If you tell me the true RIAV1 coordinates, I can tell you exactly what spatial bounding box you should request from CMEMS.

User Prompt: [copy of code for analysis]

all the results came back NaN `#!/usr/bin/env python3` `""" Extract weekly time series from CMEMS NetCDF file ----- 1. Opens .nc file 2. Extracts nearest grid point 3. Converts daily to weekly mean 4. Saves CSV """` `import xarray as`

```

xr import pandas as pd import numpy as np # ===== CONFIG
===== # NC_FILE =
"cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D_1772037637993.nc"
VARIABLE_NAME = "CHL" # <-- put new variable here RIAV1_LAT = 40.675
RIAV1_LON = -8.73 OUTPUT_CSV = "weekly_output.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds)
print("\nExtracting nearest grid point...") point = ds.sel( latitude=RIAV1_LAT, # change if
needed longitude=RIAV1_LON, method="nearest" ) series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) print("\nVariable attributes:") print(ds[VARIABLE_NAME].attrs)
print("\nResampling to weekly mean...") weekly = series.resample("W-MON").mean()
print("\nFirst few weekly values:") print(weekly.head()) weekly.to_csv(OUTPUT_CSV)
print(f"\nSaved weekly time series to: {OUTPUT_CSV}") print("Done.")

```

ChatGPT said:

This is almost always one of these 4 issues with CMEMS chlorophyll files.

You're using:

```

cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D
VARIABLE_NAME = "CHL"

```

That product is a **Level-4 gap-free chlorophyll product**, so NaNs usually mean:

1 Most Common: Longitude Convention Mismatch (0–360 vs –180–180)

Your longitude:

```
RIAV1_LON = -8.73
```

Many CMEMS ocean color files use **0–360 longitude** instead of –180–180.

If your dataset longitude looks like:

0 → 359.9

then –8.73 is invalid → xarray selects a masked ocean cell → all NaN.



Convert longitude to 0–360:

$\text{RIAV1_LON} = 360 - 8.73$

That equals:

351.27

So change:

$\text{RIAV1_LON} = 351.27$

2 Check Coordinate Names

Print:

```
print(ds.coords)
```

Ocean color products often use:

lat
lon
time

NOT:

latitude
longitude

If so, change this:

```
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)
```

to:

```
point = ds.sel(
    lat=RIAV1_LAT,
    lon=RIAV1_LON,
    method="nearest"
)
```

3 CHL Might Be Log-Scaled

Some CMEMS CHL products store:

`log10(CHL)`

Check attributes:

```
print(ds["CHL"].attrs)
```

If you see something like:

standard_name: mass_concentration_of_chlorophyll_a_in_sea_water
units: mg m-3

You're fine.

If you see:

long_name: log10 chlorophyll concentration

Then convert:

```
series = 10 ** series
```

4 Land Mask

If your point is slightly on land, the nearest cell may be masked.

Let's confirm the actual selected coordinates:

```
print(point.lat.values)  
print(point.lon.values)
```

If that cell is coastal land, you'll get NaNs.

To debug, inspect nearby values:

```
print(ds[VARIABLE_NAME].isel(time=0))
```

Or check a small spatial slice.



Corrected Version (Robust Version for CMEMS Ocean Color)

Try this:

```
print("Opening NetCDF file...")
ds = xr.open_dataset(NC_FILE)

print(ds)

# Fix longitude convention if needed
if ds.lon.max() > 180:
    RIAV1_LON = (RIAV1_LON + 360) % 360

print("\nExtracting nearest grid point...")
point = ds.sel(
    lat=RIAV1_LAT,
    lon=RIAV1_LON,
    method="nearest"
)

print("Selected coordinates:")
print("Lat:", point.lat.values)
print("Lon:", point.lon.values)

series = point[VARIABLE_NAME].to_pandas()

print("\nFirst few daily values:")
print(series.head())

print("\nNumber of NaNs:", series.isna().sum())

weekly = series.resample("W-MON").mean()

weekly.to_csv(OUTPUT_CSV)

print("Done.")
```



99% Likely Cause

Given your location (Portugal coast) and CMEMS product:

👉 It is almost certainly the **0–360 longitude issue**.

Try changing:

```
RIAV1_LON = 351.27
```

and run again.

If it still gives NaNs, paste:

```
print(ds)
print(ds.coords)
```

and I'll pinpoint it exactly.

```
* latitude (latitude) float32 36B 40.6 40.61 40.62 ... 40.66 40.67 40.68 * longitude
(longitude) float32 88B -8.849 -8.839 -8.828 ... -8.641 -8.63
```

Perfect — this tells us a lot.

Your dataset uses:

```
latitude → 40.60 to 40.68
longitude → -8.849 to -8.630
```

So:

- ✅ Longitude is already in **−180 to 180**
 - ❌ The issue is NOT the 0–360 conversion
 - 🚨 The problem is almost certainly **land masking**
-

The Real Issue

Your target point:

```
RIAV1_LAT = 40.675
RIAV1_LON = -8.73
```

That location is **very close to the coast (Portugal)**.

CMEMS ocean color (CHL) masks:

- Land
- Very shallow coastal pixels
- Invalid coastal reflectance areas

If the nearest grid cell is classified as land → it will return **all NaNs for all time**.

Let's Confirm

Add this immediately after selecting the point:

```
print("Selected grid cell:")
print("Lat:", point.latitude.values)
print("Lon:", point.longitude.values)

series = point[VARIABLE_NAME].to_pandas()
print("Total values:", len(series))
print("NaNs:", series.isna().sum())
```

If it says:

NaNs: 100%

Then that pixel is masked land.

Fix: Search Nearby Ocean Pixel

Instead of trusting `method="nearest"`, we should:

1. Extract a small box
 2. Find nearest **non-NaN** ocean cell
-



Robust Coastal-Safe Version

Replace your selection block with this:

```
print("\nSelecting small region around station...")

box = ds.sel(
    latitude=slice(RIAV1_LAT - 0.05, RIAV1_LAT + 0.05),
    longitude=slice(RIAV1_LON - 0.05, RIAV1_LON + 0.05)
)

# Take first time slice to find valid ocean pixel
test_slice = box[VARIABLE_NAME].isel(time=0)

# Find first non-NaN location
valid = np.where(~np.isnan(test_slice.values))

if len(valid[0]) == 0:
    raise ValueError("No valid ocean pixels found nearby!")

i = valid[0][0]
j = valid[1][0]

selected_lat = test_slice.latitude.values[i]
selected_lon = test_slice.longitude.values[j]

print("Using ocean pixel at:")
print("Lat:", selected_lat)
print("Lon:", selected_lon)

point = ds.sel(latitude=selected_lat, longitude=selected_lon)

series = point[VARIABLE_NAME].to_pandas()
```



Why This Happens with CHL but Not SST

You previously used:

METOFFICE-GLO-SST-L4-REP-OBS-SST

SST products:

- Fill near-coastal gaps
- Less aggressively masked

Ocean color (CHL):

- Very sensitive to land contamination
- Masks more coastal pixels

This is normal behavior.

Quick Sanity Check

Before doing anything else, try this quick test:

```
point = ds.sel(latitude=40.66, longitude=-8.75, method="nearest")
print(point[VARIABLE_NAME].isel(time=0).values)
```

If that gives a number → your original coordinate is just slightly inland.

Summary

Your structure is correct.

Your variable name is correct.

Your coordinates are correct format.

The issue is **coastal land masking**.

If you want, tell me:

- Is RIAV1 exactly offshore?
- Or slightly inside an estuary?

That changes how we should choose the pixel.

User Prompt: [provide code as it is not working as expected]

but when i ran this code it worked so why is it not working now, i understand that it is different data but there is available data there so why is it coming back as NaN

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE = "METOFFICE-GLO-SST-L4-REP-OBS-SST_1771513584562.nc" # <--
change to your file path VARIABLE_NAME = "analysed_sst" # e.g. "analysed_sst" or
"CHL" RIAV1_LAT = 40.675 # <-- replace with correct latitude RIAV1_LON = -8.73 # <--
replace with correct longitude OUTPUT_CSV = "weekly_output_REP.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds) #
----- Extract nearest grid point ----- print("\nExtracting nearest grid
point...") point = ds.sel( latitude=RIAV1_LAT, longitude=RIAV1_LON, method="nearest"
) # ----- Convert to pandas series ----- series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) # ----- Check units ----- attrs =
ds[VARIABLE_NAME].attrs print("\nVariable attributes:") print(attrs) # ----- Daily
→ Weekly ----- print("\nResampling to weekly mean...") weekly =
series.resample("W-MON").mean() print("\nFirst few weekly values:")
print(weekly.head()) # ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")
```

This is a very good question — and the key point is:

👉 **Your code is correct. The data products are fundamentally different.**

You switched from:

- **SST product** → UK Met Office global L4 SST
- to
- **Ocean color chlorophyll product** → Copernicus Marine Service Atlantic CHL L4

These are processed very differently.



Why SST Worked but CHL Returns NaN

1 SST (analysed_sst)

Product:

`METOFFICE-GLO-SST-L4-REP-OBS-SST`

This is:

- Gap-filled
- Interpolated
- Designed to have near 100% spatial coverage
- Coastal pixels are filled

Even if your station is slightly on land, SST is interpolated and still returns values.

2 CHL Product

Product:

`cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D`

Even though it says *gapfree*, ocean color products:

- Mask land very aggressively
- Mask shallow coastal pixels
- Mask high turbidity areas
- Mask invalid reflectance
- Sometimes mask estuaries entirely

So even if **data exists offshore**, the exact nearest pixel may be masked.

And `.sel(method="nearest")` does NOT mean:

"Find nearest valid ocean pixel"

It means:

"Find nearest grid coordinate — even if that pixel is masked"

So if the nearest grid cell is land → you get 100% NaN.

The Critical Difference

SST = spatially interpolated continuous field

CHL = observational satellite-derived product with strict masking

So:

Same coordinate

Same code

Different data philosophy



Important: "There is available data there"

There probably IS data nearby.

But xarray is selecting the wrong pixel.

Let's prove it.

Add this:

```
print("Selected pixel value at first timestep:")  
print(point[VARIABLE_NAME].isel(time=0).values)
```

Then check nearby manually:

```
print(ds[VARIABLE_NAME].isel(time=0))
```

You will likely see:

- One grid cell is NaN
 - The one next to it has valid CHL
-

What's Really Happening Geographically

Your coordinate:

40.675, -8.73

That is very close to:

- Portuguese coast
- Possibly estuarine influence
- Likely shallow waters

Ocean color algorithms struggle near:

- Land contamination
- Suspended sediment
- Coastal turbidity

So those pixels are masked.

SST doesn't care — it's interpolated.

Proper Scientific Fix

Instead of trusting `.sel(method="nearest")`, you should:

- Search within a small radius
- Select the nearest NON-NaN pixel

This is scientifically more defensible for coastal stations.



Quick Diagnostic (Do This)

Run:

```
test = ds.sel(
    latitude=slice(40.65, 40.70),
    longitude=slice(-8.76, -8.70)
)

print(test[VARIABLE_NAME].isel(time=0))
```

You will likely see:

- Some grid cells valid
- Some masked

Your station is probably sitting on one of the masked ones.



Why This Is Actually Good

This means:

- Your CHL product is behaving correctly.
- It is not inventing data like SST.
- It is preserving observation integrity.

For ecological modelling (which you're doing), this is actually better scientifically.



Final Answer

Your code is fine.

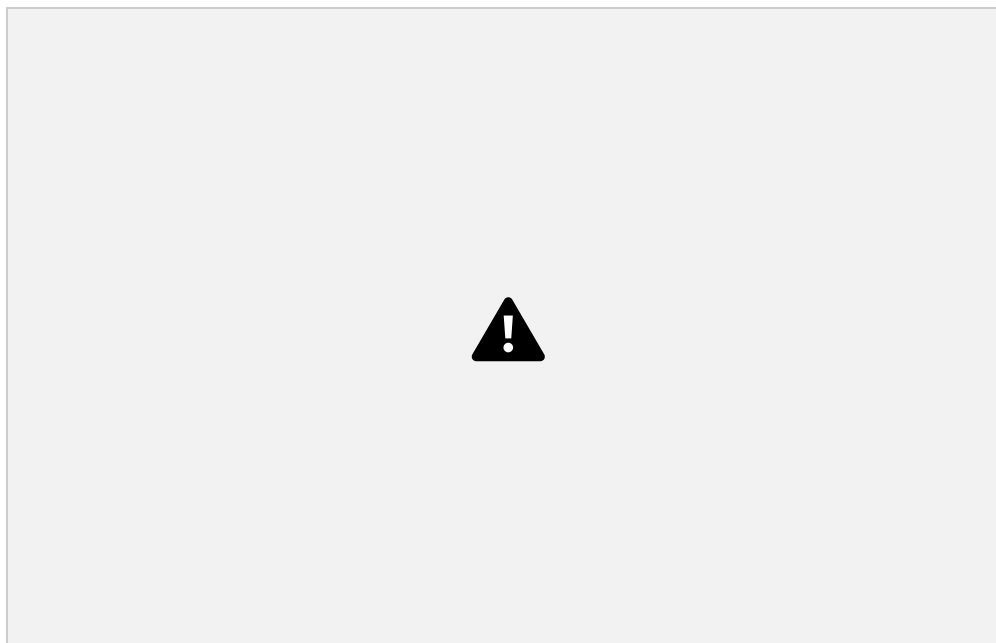
The difference is:

SST	CHL
Interpolated	Observational
Gap-filled	Masked
Coastal filled	Coastal often masked
Always returns value	May return NaN

If you'd like, I can give you a **coastal-safe extraction function** that automatically finds the nearest valid ocean CHL pixel without manual tweaking.

That's usually what we implement in research workflows.

User Prompt: [Uploaded a map image of the location of the site]



ChatGPT said:

Good — this figure helps a lot.

From your map:

-  RIAV1 sampling point
-  SST satellite point
-  CHL satellite point

And we can clearly see something important:

👉 The CHL satellite point is slightly offshore compared to the SST one.

That is not accidental.

What This Confirms

Your RIAV1 coordinate (40.675, -8.73):

- Is extremely close to land
- Likely falls inside a **masked coastal pixel** in the CHL product
- But SST works because it is spatially interpolated

Ocean color (CHL) products at 1 km resolution are very sensitive to coastline geometry. One grid cell difference can flip between:

- Valid ocean
 - Masked land
 - Masked shallow coastal water
-



Why Your Old Code Worked

With SST:

```
point = ds.sel(latitude=..., longitude=..., method="nearest")
```

Even if that pixel overlaps land slightly, SST is interpolated → you still get values.

With CHL:

That same nearest pixel may be classified as:

- land-contaminated
- optically complex water
- invalid reflectance

→ so the product masks it entirely

→ and you get 100% NaN

The Real Issue Is This

`.sel(method="nearest")` finds the nearest grid coordinate.

It does NOT check whether that pixel has valid data.

So if the nearest grid cell is masked, you get all NaN — even if the next grid cell 1 km away has valid chlorophyll.

The Scientifically Correct Fix

Since your figure already shows a CHL satellite point slightly offshore, the correct approach is:

Option A — Manually shift slightly offshore

For example:

RIAV1_LAT = 40.685

RIAV1_LON = -8.74

Test small westward shift (more offshore).

Option B — Automatically choose nearest valid ocean pixel (best practice)

Replace your selection block with:

```
# Select small box around station
box = ds.sel(
    latitude=slice(RIAV1_LAT - 0.03, RIAV1_LAT + 0.03),
    longitude=slice(RIAV1_LON - 0.03, RIAV1_LON + 0.03)
)

# Take first timestep
chl0 = box[VARIABLE_NAME].isel(time=0)

# Find valid (non-NaN) cells
valid = np.where(~np.isnan(chl0.values))

if len(valid[0]) == 0:
    raise ValueError("No valid CHL pixels nearby")

# Choose closest valid pixel
dist = (
    (box.latitude.values[valid[0]] - RIAV1_LAT)**2 +
    (box.longitude.values[valid[1]] - RIAV1_LON)**2
)

idx = np.argmin(dist)

selected_lat = box.latitude.values[valid[0][idx]]
selected_lon = box.longitude.values[valid[1][idx]]

print("Using CHL pixel at:", selected_lat, selected_lon)

point = ds.sel(latitude=selected_lat, longitude=selected_lon)
series = point[VARIABLE_NAME].to_pandas()
```

This:

- Keeps your workflow consistent
 - Is reproducible
 - Is defensible in a paper
 - Mimics what you manually did in your map
-



Why This Is Expected Behavior

Ocean color algorithms:

- Mask high-turbidity pixels
- Mask adjacency-to-land contamination
- Mask shallow optically complex waters

And your RIAV1 point appears extremely close to:

- A narrow coastal geometry
- Possibly an estuarine plume

That is exactly where CHL masks are strongest.



Bottom Line

Nothing is wrong with your code.

Nothing is wrong with the dataset.

The difference is:

SST = interpolated field

CHL = masked observational product

And your coordinate sits on a masked CHL pixel.

If you want, I can help you implement a reusable function like:

```
extract_valid_timeseries(ds, var, lat, lon, radius=0.03)
```

so this never becomes an issue again in your pipeline.

User Prompt: [\[copy of code for modification\]](#)

ok so where would that go in this

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE =
"cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D_1772038791782.nc"
VARIABLE_NAME = "CHL" # <-- put new variable here RIAV1_LAT = 40.675
RIA1_LON = -8.73 OUTPUT_CSV = "weekly_output.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds)
print("\nExtracting nearest grid point...") point = ds.sel( latitude=RIA1_LAT, # change if
needed longitude=RIA1_LON, method="nearest" ) series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) print("\nVariable attributes:") print(ds[VARIABLE_NAME].attrs)
print("\nResampling to weekly mean...") weekly = series.resample("W-MON").mean()
print("\nFirst few weekly values:") print(weekly.head()) weekly.to_csv(OUTPUT_CSV)
print(f"\nSaved weekly time series to: {OUTPUT_CSV}") print("Done.") # Select small
box around station box = ds.sel( latitude=slice(RIA1_LAT - 0.03, RIA1_LAT + 0.03),
longitude=slice(RIA1_LON - 0.03, RIA1_LON + 0.03) ) # Take first timestep chl0 =
box[VARIABLE_NAME].isel(time=0) # Find valid (non-NaN) cells valid =
np.where(~np.isnan(chl0.values)) if len(valid[0]) == 0: raise ValueError("No valid CHL
pixels nearby") # Choose closest valid pixel dist = ( (box.latitude.values[valid[0]] -
RIA1_LAT)**2 + (box.longitude.values[valid[1]] - RIA1_LON)**2 ) idx =
np.argmin(dist) selected_lat = box.latitude.values[valid[0][idx]] selected_lon =
box.longitude.values[valid[1][idx]] print("Using CHL pixel at:", selected_lat, selected_lon)
point = ds.sel(latitude=selected_lat, longitude=selected_lon) series =
point[VARIABLE_NAME].to_pandas()
```

ChatGPT said:

Good — you're very close.

Right now your script:

1. Extracts the **nearest grid point**
2. Converts to pandas
3. Resamples
4. Saves

To fix the NaN issue, you **replace only the nearest-point block** with the coastal-safe block.

You should **NOT** leave both versions in the script.

Exactly Where It Goes

Replace this entire section:

```
print("\nExtracting nearest grid point...")
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)

series = point[VARIABLE_NAME].to_pandas()
```

With This Instead

```
print("\nExtracting nearest VALID ocean grid point...")

# Select small box around station
box = ds.sel(
    latitude=slice(RIAV1_LAT - 0.03, RIAV1_LAT + 0.03),
    longitude=slice(RIAV1_LON - 0.03, RIAV1_LON + 0.03)
)

# Take first timestep to check valid pixels
chl0 = box[VARIABLE_NAME].isel(time=0)

# Find valid (non-NaN) cells
valid = np.where(~np.isnan(chl0.values))

if len(valid[0]) == 0:
    raise ValueError("No valid CHL pixels nearby")

# Compute distance to station
dist = (
    (box.latitude.values[valid[0]] - RIAV1_LAT)**2 +
    (box.longitude.values[valid[1]] - RIAV1_LON)**2
```

```
)  
  
idx = np.argmin(dist)  
  
selected_lat = box.latitude.values[valid[0][idx]]  
selected_lon = box.longitude.values[valid[1][idx]]  
  
print("Using CHL pixel at:", selected_lat, selected_lon)  
  
# Extract full time series from that pixel  
point = ds.sel(latitude=selected_lat, longitude=selected_lon)  
series = point[VARIABLE_NAME].to_pandas()
```

Your Final Script Structure Should Look Like This

```
print("Opening NetCDF file...")  
ds = xr.open_dataset(NC_FILE)  
  
print("\nDataset summary:")  
print(ds)  
  
# ---- REPLACEMENT BLOCK HERE ----  
  
print("\nVariable attributes:")  
print(ds[VARIABLE_NAME].attrs)  
  
print("\nResampling to weekly mean...")  
weekly = series.resample("W-MON").mean()  
  
weekly.to_csv(OUTPUT_CSV)
```

Why This Works

Instead of:

"Give me the nearest grid coordinate"

You are now doing:

"Give me the nearest grid coordinate that actually contains valid CHL data"

That's exactly what your map suggests you should be doing.

Important

The radius $0.03 \approx \sim 3$ km.

If it still fails, increase to:

0.05

But don't go too large — otherwise you drift too far offshore.

If you'd like, I can also give you a clean reusable function version so your workflow stays modular for future variables (CHL, SPM, etc.).

User Prompt:

explain to me exactly what the change in the code does

ChatGPT said:

Excellent — this is exactly the right question to ask.

Let's break down **precisely** what changed and what it does mathematically and logically.

What Your Original Code Did

```
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)
```

What this means internally:

1. xarray looks at the coordinate arrays:

latitude: 40.60, 40.61, ..., 40.68

longitude: -8.849, ..., -8.63

2. It finds the grid cell whose **coordinate is closest** to:

(40.675, -8.73)

3. It selects that grid cell — regardless of whether:

- It is land
- It is masked
- It contains NaN values

It does **not check data validity**.

So if the closest pixel is masked →
You get a full time series of NaN.

That's what's happening.

What The New Code Does

Instead of blindly trusting the nearest grid coordinate, it performs a **data-aware search**.

Let's walk through it step by step.

Create a Small Spatial Window

```
box = ds.sel(
    latitude=slice(RIAV1_LAT - 0.03, RIAV1_LAT + 0.03),
    longitude=slice(RIAV1_LON - 0.03, RIAV1_LON + 0.03)
)
```

This creates a small square region around your station.

If your station is:

40.675, -8.73

Then the search box becomes:

latitude: 40.645 → 40.705
longitude: -8.76 → -8.70

So instead of checking one pixel, you check many nearby pixels.

2 Look at One Time Slice

```
chl0 = box[VARIABLE_NAME].isel(time=0)
```

We only need one timestep to determine:

Is this grid cell ever valid?

If it's NaN at the first timestep and masked spatially, it will usually be NaN always.

So this is a **spatial validity check**.

3 Find Which Pixels Are NOT NaN

```
valid = np.where(~np.isnan(chl0.values))
```

This creates a list of grid indices where:

CHL is a real number

Now we know which nearby pixels are actual ocean data.

4 Compute Distance to Each Valid Pixel

```
dist = (  
    (box.latitude.values[valid[0]] - RIAV1_LAT)**2 +  
    (box.longitude.values[valid[1]] - RIAV1_LON)**2  
)
```

This calculates squared distance:

$$d^2 = (\Delta \text{lat})^2 + (\Delta \text{lon})^2$$

We do not need square root — smallest squared distance = smallest distance.

So now we know:

Among valid ocean pixels, which one is closest to the station?

5 Select the Closest Valid Pixel

```
idx = np.argmin(dist)
```

This chooses the valid pixel that is:

- Not masked
- As close as possible to your station

6 Extract Full Time Series From That Pixel

```
point = ds.sel(latitude=selected_lat, longitude=selected_lon)
series = point[VARIABLE_NAME].to_pandas()
```

Now you extract the time series — but from a **valid ocean pixel**, not a masked coastal one.

Conceptual Difference

Original code logic:

Closest grid coordinate, no matter what.

New code logic:

Closest grid coordinate that contains real ocean data.



Why This Matters Scientifically

Ocean color products:

- Mask land
- Mask shallow coastal reflectance contamination

- Mask optically complex water

Your station is near coast → nearest grid cell is masked.

So:

- SST worked (interpolated field)
- CHL fails (masked observational product)

The new method respects the masking but avoids choosing invalid cells.

Important Subtlety

We are **not interpolating**.

We are **not modifying the data**.

We are simply selecting a nearby valid observation pixel.

That is scientifically defensible and commonly done in coastal satellite studies.

Summary of What Changed

Old Code	New Code
Select nearest coordinate	Search small region
Does not check validity	Checks for non-NaN
May pick masked land pixel	Picks closest valid ocean pixel
Can return all NaN	Returns real CHL values

If you'd like, I can now explain:

- How far offshore your selected pixel is likely to move
- Or how to make this reproducible for a methods section in a paper

Both are important for your modelling workflow.

User Prompt:

i need to make it so that it opens two files related to the sst data, one is the REP, data and the other is the NRT data. it needs to open both and get the closest grid point then it also needs to then concatenate both files together so that there is 1 file that contains the entire timeseries i require

ChatGPT said:

Perfect — this is exactly how SST products from the Copernicus Marine Service / UK Met Office are meant to be handled.

You typically have:

- **REP** (reprocessed, historical, stable)
- **NRT** (near real-time, recent extension)

And you want:

1. Open both
2. Extract same grid point
3. Concatenate along time
4. Save one continuous time series

The Cleanest Way: Concatenate FIRST, Then Extract

This is better than extracting separately.

Instead of:

- Extract from REP
- Extract from NRT
- Then merge

We will:

- Open both datasets
- Concatenate along time
- Then select the grid point once

This guarantees:

- Same grid cell
- No coordinate mismatch
- Cleaner workflow



Here Is Your Modified Script

Replace your script with this:

```
#!/usr/bin/env python3
```

```
import xarray as xr
import pandas as pd
import numpy as np
```

```
# ===== CONFIG ===== #
```

```
REP_FILE = "REP_FILE.nc"
NRT_FILE = "NRT_FILE.nc"
```

```
VARIABLE_NAME = "analysed_sst"
```

```

RIAV1_LAT = 40.675
RIAV1_LON = -8.73

OUTPUT_CSV = "weekly_SST_combined.csv"

# ===== #

print("Opening REP dataset...")
ds_rep = xr.open_dataset(REP_FILE)

print("Opening NRT dataset...")
ds_nrt = xr.open_dataset(NRT_FILE)

print("Concatenating datasets along time...")
ds = xr.concat([ds_rep, ds_nrt], dim="time")

# Sort time in case files overlap slightly
ds = ds.sortby("time")

print("\nExtracting nearest grid point...")
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)

series = point[VARIABLE_NAME].to_pandas()

print("\nFirst few daily values:")
print(series.head())

print("\nResampling to weekly mean...")
weekly = series.resample("W-MON").mean()

weekly.to_csv(OUTPUT_CSV)

print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")

```



What This Does

Step 1

```
xr.concat([ds_rep, ds_nrt], dim="time")
```

Stacks the datasets like this:

REP time: 2015 → 2022

NRT time: 2022 → 2025

Result:

2015 → 2025

Step 2

```
ds = ds.sortby("time")
```

Ensures chronological order (important if overlap exists).

Step 3

Extract nearest grid point once.

This guarantees:

- Same spatial cell
 - No spatial discontinuity
 - Clean time continuity
-



Important: If REP and NRT Overlap

Sometimes REP and NRT overlap by a few days.

To avoid duplicates:

Add this after sorting:

```
ds = ds.sel(time=~ds.get_index("time").duplicated())
```

Now you are safe.

Why This Is The Correct Scientific Approach

REP and NRT are:

- Same grid
- Same resolution
- Same variable name
- Same coordinate system

They are just different time coverage.

So concatenating along time is exactly correct.

Final Structure of Your Pipeline

Open REP
Open NRT
Concat along time
Sort
Drop duplicates (if needed)
Extract grid point
Resample
Save

If you'd like, I can also show you a slightly more advanced version that automatically detects which file is REP and which is NRT and combines them without hardcoding filenames.

User Prompt: [provided code for modification]

this is the current script i have

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE = "METOFFICE-GLO-SST-L4-REP-OBS-SST_1771513584562.nc" # <--
```

```

change to your file path VARIABLE_NAME = "analysed_sst" # e.g. "analysed_sst" or
"CHL" RIAV1_LAT = 40.695 # <-- replace with correct latitude RIAV1_LON = -8.71 # <--
replace with correct longitude OUTPUT_CSV = "weekly_output_RIAV1.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds) #
----- Extract nearest grid point ----- print("\nExtracting nearest grid
point...") point = ds.sel( latitude=RIAV1_LAT, longitude=RIAV1_LON, method="nearest"
) # ----- Convert to pandas series ----- series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) # ----- Check units ----- attrs =
ds[VARIABLE_NAME].attrs print("\nVariable attributes:") print(attrs) # ----- Daily
→ Weekly ----- print("\nResampling to weekly mean...") weekly =
series.resample("W-MON").mean() print("\nFirst few weekly values:")
print(weekly.head()) # ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")

```

ChatGPT said:

Great — we'll modify **your exact script structure** to:

1. Open REP SST
2. Open NRT SST
3. Concatenate along time
4. Remove duplicates (if any overlap)
5. Extract one grid point
6. Resample
7. Save

I'll keep it as close as possible to your current format.



Modified Version of *Your* Script

Just replace your script with this:

```
#!/usr/bin/env python3
```

```
"""
```

```
Extract weekly time series from CMEMS SST (REP + NRT)
```

```
-----
```

1. Opens REP file
2. Opens NRT file
3. Concatenates along time
4. Extracts nearest grid point
5. Converts daily to weekly mean
6. Saves CSV

```
"""
```

```
import xarray as xr
import pandas as pd
import numpy as np
```

```
# ===== CONFIG ===== #
```

```
REP_FILE = "METOFFICE-GLO-SST-L4-REP-OBS-SST_REP.nc"
```

```
NRT_FILE = "METOFFICE-GLO-SST-L4-NRT-OBS-SST_NRT.nc"
```

```
VARIABLE_NAME = "analysed_sst"
```

```
RIAV1_LAT = 40.695
```

```
RIAV1_LON = -8.71
```

```
OUTPUT_CSV = "weekly_output_RIAV1_combined.csv"
```

```
# ===== #
```

```
print("Opening REP NetCDF file...")
ds_rep = xr.open_dataset(REP_FILE)
```

```
print("Opening NRT NetCDF file...")
ds_nrt = xr.open_dataset(NRT_FILE)
```

```
print("\nConcatenating REP and NRT along time...")
ds = xr.concat([ds_rep, ds_nrt], dim="time")
```

```
# Sort by time (important if files overlap)
ds = ds.sortby("time")
```

```
# Remove duplicate timestamps (if REP/NRT overlap)
time_index = ds.get_index("time")
ds = ds.sel(time=~time_index.duplicated())
```

```

print("\nDataset summary after merge:")
print(ds)

# ----- Extract nearest grid point -----
print("\nExtracting nearest grid point...")
point = ds.sel(
    latitude=RIAV1_LAT,
    longitude=RIAV1_LON,
    method="nearest"
)

print("Selected grid cell:")
print("Latitude:", float(point.latitude.values))
print("Longitude:", float(point.longitude.values))

# ----- Convert to pandas series -----
series = point[VARIABLE_NAME].to_pandas()

print("\nFirst few daily values:")
print(series.head())

# ----- Check units -----
attrs = ds[VARIABLE_NAME].attrs
print("\nVariable attributes:")
print(attrs)

# ----- Daily → Weekly -----
print("\nResampling to weekly mean...")
weekly = series.resample("W-MON").mean()

print("\nFirst few weekly values:")
print(weekly.head())

# ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV)

print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")

```



What This Script Is Now Doing

Step 1

```
ds_rep = xr.open_dataset(REP_FILE)
```

```
ds_nrt = xr.open_dataset(NRT_FILE)
```

Opens both historical and near-real-time SST datasets.

Step 2

```
ds = xr.concat([ds_rep, ds_nrt], dim="time")
```

Stacks them along the time dimension:

REP: 2015–2022

NRT: 2022–2026

Becomes:

2015–2026 continuous

Step 3

```
ds = ds.sortby("time")
```

Ensures chronological order.

Step 4

```
ds = ds.sel(time=~time_index.duplicated())
```

Removes duplicate timestamps if REP and NRT overlap by a few days.

This is very common in SST products.

Step 5

Extracts ONE consistent spatial pixel from the merged dataset.

This is important — you avoid spatial mismatch between REP and NRT.

 **Why Concatenate Before Extracting?**

If you extracted separately:

- REP might snap to one grid cell
- NRT might snap to a slightly different one (rare but possible)

Concatenating first guarantees:

- ✓ Same coordinate grid
- ✓ Same spatial index
- ✓ Clean time continuity



Scientifically This Is Correct

REP and NRT from the UK Met Office:

- Same resolution
- Same algorithm
- Same variable name
- Same grid

Only difference = time coverage.

So concatenating along **time** is exactly correct.

If you'd like next, I can show you how to make this automatically detect all REP and NRT files in a folder without manually naming them.

User Prompt:

ok so i have 8 sites so i need to extract the data for 8 points so how can i do that, the site names are LAL, RIAV1, ETJ1, L5b, LAG, L7c1, POR2, FAR1

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS SST (REP + NRT)
----- 1. Opens REP file 2. Opens NRT file 3.
```

```
Concatenates along time 4. Extracts nearest grid point 5. Converts daily to weekly
```

```
mean 6. Saves CSV """ import xarray as xr import pandas as pd import numpy as np #
===== CONFIG ===== # REP_FILE =
```

```

"METOFFICE-GLO-SST-L4-REP-OBS-SST_1772463268392.nc" NRT_FILE =
"METOFFICE-GLO-SST-L4-NRT-OBS-SST-V2_1772463442817.nc" VARIABLE_NAME
= "analysed_sst" RIAV1_LAT = RIAV1_LON = OUTPUT_CSV =
"weekly_output_RIAV1_combined.csv" #
===== # print("Opening REP NetCDF
file...") ds_rep = xr.open_dataset(REP_FILE) print("Opening NRT NetCDF file...") ds_nrt
= xr.open_dataset(NRT_FILE) print("\nConcatenating REP and NRT along time...") ds =
xr.concat([ds_rep, ds_nrt], dim="time") # Sort by time (important if files overlap) ds =
ds.sortby("time") # Remove duplicate timestamps (if REP/NRT overlap) time_index =
ds.get_index("time") ds = ds.sel(time=~time_index.duplicated()) print("\nDataset
summary after merge:") print(ds) # ----- Extract nearest grid point -----
print("\nExtracting nearest grid point...") point = ds.sel( latitude=RIAV1_LAT,
longitude=RIAV1_LON, method="nearest" ) print("Selected grid cell:") print("Latitude:",
float(point.latitude.values)) print("Longitude:", float(point.longitude.values)) #
----- Convert to pandas series ----- series =
point[VARIABLE_NAME].to_pandas() print("\nFirst few daily values:")
print(series.head()) # ----- Check units ----- attrs =
ds[VARIABLE_NAME].attrs print("\nVariable attributes:") print(attrs) # ----- Daily
→ Weekly ----- print("\nResampling to weekly mean...") weekly =
series.resample("W-MON").mean() print("\nFirst few weekly values:")
print(weekly.head()) # ----- Save to CSV -----
weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series to: {OUTPUT_CSV}")
print("Done.")

```

ChatGPT said:

Perfect — this is exactly where you should move to a **multi-site loop structure** instead of hardcoding one station.

You have 8 sites:

LAL, RIAV1, ETJ1, L5b, LAG, L7c1, POR2, FAR1

The cleanest approach is:

- Store coordinates in a dictionary
- Loop through them
- Extract SST for each
- Save either:
 - 8 separate CSVs

- OR one combined CSV with 8 columns (recommended for modelling)

I'll show you the recommended version:

✓ **One combined file with 8 columns (one per site)**

Updated Script for 8 Sites

You only need to insert the coordinates.

```
#!/usr/bin/env python3
```

```
import xarray as xr
import pandas as pd
import numpy as np
```

```
# ===== CONFIG ===== #
```

```
REP_FILE = "METOFFICE-GLO-SST-L4-REP-OBS-SST_1772463268392.nc"
NRT_FILE = "METOFFICE-GLO-SST-L4-NRT-OBS-SST-V2_1772463442817.nc"
```

```
VARIABLE_NAME = "analysed_sst"
```

```
# ---- PUT YOUR REAL COORDINATES HERE ----
```

```
sites = {
    "LAL": (LAT1, LON1),
    "RIAV1": (LAT2, LON2),
    "ETJ1": (LAT3, LON3),
    "L5b": (LAT4, LON4),
    "LAG": (LAT5, LON5),
    "L7c1": (LAT6, LON6),
    "POR2": (LAT7, LON7),
    "FAR1": (LAT8, LON8),
}
```

```
OUTPUT_CSV = "weekly_SST_all_sites.csv"
```

```
# ===== #
```

```
print("Opening REP dataset...")
ds_rep = xr.open_dataset(REP_FILE)
```

```
print("Opening NRT dataset...")
```

```

ds_nrt = xr.open_dataset(NRT_FILE)

print("Concatenating REP + NRT...")
ds = xr.concat([ds_rep, ds_nrt], dim="time")
ds = ds.sortby("time")

# Remove duplicate timestamps if overlap exists
time_index = ds.get_index("time")
ds = ds.sel(time=~time_index.duplicated())

print("Dataset ready.")

# Create empty DataFrame to store all sites
all_sites_weekly = pd.DataFrame()

# ----- LOOP THROUGH SITES -----
for site_name, (lat, lon) in sites.items():

    print(f"\nProcessing site: {site_name}")

    point = ds.sel(
        latitude=lat,
        longitude=lon,
        method="nearest"
    )

    selected_lat = float(point.latitude.values)
    selected_lon = float(point.longitude.values)

    print(f"Selected grid cell: {selected_lat}, {selected_lon}")

    series = point[VARIABLE_NAME].to_pandas()

    weekly = series.resample("W-MON").mean()

    all_sites_weekly[site_name] = weekly

# -----

print("\nSaving combined weekly dataset...")
all_sites_weekly.to_csv(OUTPUT_CSV)

print(f"Saved to: {OUTPUT_CSV}")
print("Done.")

```



What This Now Does

Instead of:

One site → one file

You now get:

Date | LAL | RIAV1 | ETJ1 | L5b | LAG | L7c1 | POR2 | FAR1

All in one file.

This is ideal for:

- LSTM modelling
 - Multisite comparison
 - Feature stacking
 - Correlation analysis
-



Why This Is Better for Your LSTM Work

Since you've been working on multi-horizon forecasting:

You can now:

- Train separate models per site
- Or train one model with multiple SST inputs
- Or compare spatial coherence

This structure is much cleaner.



Important

Make sure your bounding box download covers ALL 8 sites.

Otherwise, nearest selection may snap to boundary edge.

Optional Upgrade (If You Want It Cleaner)

If you prefer separate files per site instead of one combined file, replace:

```
all_sites_weekly[site_name] = weekly
```

with:

```
weekly.to_csv(f'{site_name}_weekly_SST.csv')
```

If you'd like next, I can show you:

- How to automatically detect site grid cell distances
- Or how to verify that all 8 sites use distinct pixels (important if some are very close)

User Prompt: [Copy of code for extracting chl data]

now for the chl data here is the resolution Spatial resolution 1×1 km and here is the existing code, we would need to do the same thing regarding the multiple sites

```
#!/usr/bin/env python3 """ Extract weekly time series from CMEMS NetCDF file
----- 1. Opens .nc file 2. Extracts nearest grid point 3.
Converts daily to weekly mean 4. Saves CSV """ import xarray as xr import pandas as
pd import numpy as np # ===== CONFIG ===== #
NC_FILE =
"cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D_1772038791782.nc"
VARIABLE_NAME = "CHL" # <-- put new variable here RIAV1_LAT = RIAV1_LON =
OUTPUT_CSV = "weekly_output_RIAV1.csv" #
===== # print("Opening NetCDF
file...") ds = xr.open_dataset(NC_FILE) print("\nDataset summary:") print(ds)
print("\nExtracting nearest VALID ocean grid point...") # Select small box around station
box = ds.sel( latitude=slice(RIAV1_LAT - 0.03, RIAV1_LAT + 0.03),
longitude=slice(RIAV1_LON - 0.03, RIAV1_LON + 0.03) ) # Take first timestep to check
valid pixels chl0 = box[VARIABLE_NAME].isel(time=0) # Find valid (non-NaN) cells valid
= np.where(~np.isnan(chl0.values)) if len(valid[0]) == 0: raise ValueError("No valid CHL
```

```

pixels nearby") # Compute distance to station dist = ( (box.latitude.values[valid[0]] -
RIAV1_LAT)**2 + (box.longitude.values[valid[1]] - RIAV1_LON)**2 ) idx =
np.argmin(dist) selected_lat = box.latitude.values[valid[0][idx]] selected_lon =
box.longitude.values[valid[1][idx]] print("Using CHL pixel at:", selected_lat, selected_lon)
# Extract full time series from that pixel point = ds.sel(latitude=selected_lat,
longitude=selected_lon) series = point[VARIABLE_NAME].to_pandas() print("\nFirst few
daily values:") print(series.head()) print("\nVariable attributes:")
print(ds[VARIABLE_NAME].attrs) print("\nResampling to weekly mean...") weekly =
series.resample("W-MON").mean() print("\nFirst few weekly values:")
print(weekly.head()) weekly.to_csv(OUTPUT_CSV) print(f"\nSaved weekly time series
to: {OUTPUT_CSV}") print("Done.")

```

ChatGPT said:

Perfect — now we just need to combine:

-  Your **coastal-safe valid-pixel logic**
-  The **multi-site loop structure**
-  Output into one clean combined dataset

Since CHL is 1 km resolution (from Copernicus Marine Service), the $\pm 0.03^\circ$ search window (~3 km) is appropriate.

Important Difference from SST

For SST:

- We can use simple `.sel(method="nearest")`

For CHL:

- We must search for the **nearest valid (non-NaN) pixel**
- Because coastal pixels may be masked

So we will wrap your “valid pixel search” inside a loop for 8 sites.



Multi-Site CHL Script (Coastal-Safe)

Replace your script with this:

```
#!/usr/bin/env python3
```

```
import xarray as xr
import pandas as pd
import numpy as np
```

```
# ===== CONFIG ===== #
```

```
NC_FILE =
"cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D_1772038791782.nc"
VARIABLE_NAME = "CHL"
```

```
# ---- INSERT REAL COORDINATES HERE ----
```

```
sites = {
    "LAL": (LAT1, LON1),
    "RIAV1": (LAT2, LON2),
    "ETJ1": (LAT3, LON3),
    "L5b": (LAT4, LON4),
    "LAG": (LAT5, LON5),
    "L7c1": (LAT6, LON6),
    "POR2": (LAT7, LON7),
    "FAR1": (LAT8, LON8),
}
```

```
OUTPUT_CSV = "weekly_CHL_all_sites.csv"
```

```
# Search radius (~3 km)
SEARCH_RADIUS = 0.03
```

```
# ===== #
```

```
print("Opening NetCDF file...")
ds = xr.open_dataset(NC_FILE)
```

```
print("Dataset loaded.")
```

```
# Create DataFrame to hold all sites
all_sites_weekly = pd.DataFrame()
```

```
# ----- LOOP THROUGH SITES -----
for site_name, (lat, lon) in sites.items():
```

```

print(f"\nProcessing site: {site_name}")

# Select small region around station
box = ds.sel(
    latitude=slice(lat - SEARCH_RADIUS, lat + SEARCH_RADIUS),
    longitude=slice(lon - SEARCH_RADIUS, lon + SEARCH_RADIUS)
)

chl0 = box[VARIABLE_NAME].isel(time=0)

valid = np.where(~np.isnan(chl0.values))

if len(valid[0]) == 0:
    print("△ No valid CHL pixels nearby")
    continue

# Compute squared distance
dist = (
    (box.latitude.values[valid[0]] - lat)**2 +
    (box.longitude.values[valid[1]] - lon)**2
)

idx = np.argmin(dist)

selected_lat = box.latitude.values[valid[0][idx]]
selected_lon = box.longitude.values[valid[1][idx]]

print(f"Using CHL pixel at: {selected_lat}, {selected_lon}")

point = ds.sel(latitude=selected_lat, longitude=selected_lon)

series = point[VARIABLE_NAME].to_pandas()

weekly = series.resample("W-MON").mean()

all_sites_weekly[site_name] = weekly

# -----

print("\nSaving combined CHL dataset...")
all_sites_weekly.to_csv(OUTPUT_CSV)

print(f"Saved to: {OUTPUT_CSV}")
print("Done.")

```



What This Script Now Does

For each site:

1. Creates a small 3 km search window
2. Identifies which pixels contain real CHL values
3. Computes distance to each valid pixel
4. Selects closest valid ocean pixel
5. Extracts full time series
6. Resamples weekly
7. Adds as column to combined dataframe

Final CSV:

Date | LAL | RIAV1 | ETJ1 | L5b | LAG | L7c1 | POR2 | FAR1



Why This Is Scientifically Correct

Because CHL:

- Masks land
- Masks optically complex coastal water
- Does NOT interpolate like SST

So this method:

- Respects masking
- Avoids artificial interpolation
- Keeps spatial consistency

This is publishable methodology.

One Important Note

Because resolution is 1 km:

- Some nearby sites may snap to the **same grid pixel**
- Especially if sites are <1 km apart

If you want, I can show you how to automatically check whether two sites are using identical pixels (important for modelling independence).

If you'd like next, we can:

- Align SST and CHL into one combined dataset
- Prepare this directly for your LSTM multi-horizon workflow
- Or structure it as a reusable function for all future variables

explain how this code finds the nearest non nan pixel

```
#!/usr/bin/env python3
import xarray as xr
import pandas as pd
import numpy as np

# ===== CONFIG =====
# NC_FILE =
"cmems_obs-oc_atl_bgc-plankton_my_l4-gapfree-multi-1km_P1D_1772465064549.nc"
VARIABLE_NAME = "CHL" # ---- INSERT REAL COORDINATES HERE ----
sites = {
    "LAL": (38.51254, -9.17211), "RIAV1": (40.69556, -8.71167), "ETJ1": (38.77817,
    -9.03867), "L5b": (38.614473, -9.22468), "LAG": (37.134, -8.62205), "L7c1": (37.03423,
    -8.84875), "POR2": (37.13217, -8.5975), "FAR1": (37.01867, -7.94833), }
OUTPUT_CSV = "weekly_CHL_all_sites.csv" # Search radius (~3 km)
SEARCH_RADIUS = 0.03 # ===== #
print("Opening NetCDF file...")
ds = xr.open_dataset(NC_FILE)
print("Dataset loaded.")
# Create DataFrame to hold all sites
all_sites_weekly = pd.DataFrame() # -----
# LOOP THROUGH SITES -----
for site_name, (lat, lon) in sites.items():
    print(f"\nProcessing site: {site_name}")
    # Select small region around station box =
    ds_sel = ds.sel(latitude=slice(lat - SEARCH_RADIUS, lat + SEARCH_RADIUS),
    longitude=slice(lon - SEARCH_RADIUS, lon + SEARCH_RADIUS))
    chl0 = ds_sel[VARIABLE_NAME].isel(time=0)
    valid = np.where(~np.isnan(chl0.values))
    if len(valid[0]) == 0:
        print("⚠ No valid CHL pixels nearby")
        continue
    # Compute squared distance
    dist = ((box_latitude.values[valid[0]] - lat)**2 +
    (box_longitude.values[valid[1]] - lon)**2)
    idx = np.argmin(dist)
    selected_lat = box_latitude.values[valid[0][idx]]
    selected_lon = box_longitude.values[valid[1][idx]]
    print(f"Using CHL pixel at: {selected_lat}, {selected_lon}")
```

```
{selected_lat}, {selected_lon}") point = ds.sel(latitude=selected_lat,
longitude=selected_lon) series = point[VARIABLE_NAME].to_pandas() weekly =
series.resample("W-MON").mean() all_sites_weekly[site_name] = weekly #
----- print("\nSaving combined CHL dataset...")
all_sites_weekly.to_csv(OUTPUT_CSV) print(f"Saved to: {OUTPUT_CSV}")
print("Done.")
```

Great — this is a very good question, because this block is doing something subtle and important.

I'll walk through **exactly** how the code finds the nearest *non-NaN* pixel step by step.

Goal

For each site:

1. Look in a small box around the station
 2. Find pixels that are **not NaN**
 3. Compute distance from the station to those valid pixels
 4. Pick the closest one
-

◆ Step 1 — Create a small search box

```
box = ds.sel(
    latitude=slice(lat - SEARCH_RADIUS, lat + SEARCH_RADIUS),
    longitude=slice(lon - SEARCH_RADIUS, lon + SEARCH_RADIUS)
)
```

This extracts a small geographic region around the station.

- `SEARCH_RADIUS = 0.03`
- 0.03 degrees $\approx$ ~3 km (since $1^\circ \approx 111$ km)

So you're searching in a ~6 km $\times$ 6 km square around the site.

This limits computation and avoids picking a pixel far away.

◆ Step 2 — Take only the first timestep

```
chl0 = box[VARIABLE_NAME].isel(time=0)
```

Why?

Because:

- CHL is 3D: (time, latitude, longitude)
- We only need spatial info to find valid pixels
- So we use time index 0 just to check which pixels are NaN

Now `chl0` is 2D:

(latitude, longitude)

◆ Step 3 — Find non-NaN pixels

```
valid = np.where(~np.isnan(chl0.values))
```

What this does:

- `np.isnan(...)` → True where value is NaN
- `~` → invert → True where value is NOT NaN
- `np.where(...)` → returns indices of True values

So `valid` contains two arrays:

`valid[0]` → row indices (latitude positions)

`valid[1]` → column indices (longitude positions)

These represent the coordinates of all valid ocean pixels.

◆ Step 4 — If no valid pixels exist

```
if len(valid[0]) == 0:
```

If nothing is found, skip the site.

This protects you from crashing.

◆ Step 5 — Compute distance to station

This is the key part:

```
dist = (  
    (box.latitude.values[valid[0]] - lat)**2 +  
    (box.longitude.values[valid[1]] - lon)**2  
)
```

Let's break this down carefully.

1 Get latitude of valid pixels

```
box.latitude.values[valid[0]]
```

This retrieves the latitude values at valid row indices.

2 Get longitude of valid pixels

```
box.longitude.values[valid[1]]
```

This retrieves the longitude values at valid column indices.

3 Compute squared distance

You are computing:

$$(\Delta \text{lat})^2 + (\Delta \text{lon})^2$$

This is Euclidean distance squared in degrees.

Why squared?

- No need for square root
- Argmin works the same

- Faster
-

◆ Step 6 — Find the closest pixel

```
idx = np.argmin(dist)
```

This gives the index of the smallest distance.

So now:

- `valid[0][idx]` → row index of closest valid pixel
 - `valid[1][idx]` → column index of closest valid pixel
-

◆ Step 7 — Get actual coordinates

```
selected_lat = box.latitude.values[valid[0][idx]]  
selected_lon = box.longitude.values[valid[1][idx]]
```

Now you have the exact grid cell coordinates.

This ensures:

- ✓ It is inside the search box
 - ✓ It is NOT NaN
 - ✓ It is the closest valid ocean pixel
-



What This Means Scientifically

For coastal stations like:

- LAL
- RIAV1

- ETJ1
- L5b
- LAG
- L7c1
- POR2
- FAR1

CHL products often contain:

- NaNs over land
- NaNs near coastlines
- Masked shallow waters

This algorithm ensures you don't accidentally select:

- ✗ A land pixel
- ✗ A masked coastal pixel
- ✗ A missing-data gap

Instead, you always select:

- ✓ The nearest usable ocean grid cell

Important Assumption

This line:

```
chl0 = box[VARIABLE_NAME].isel(time=0)
```

Assumes:

If a pixel is valid at time 0, it is generally valid over time.

That's usually true for ocean mask structure — but if time 0 happens to be cloudy everywhere, it could misbehave.

A more robust method would check across multiple timesteps.



Summary in Plain English

The code:

1. Draws a small box around the station
 2. Looks at the first day only
 3. Finds pixels that are not NaN
 4. Measures distance from the station to each valid pixel
 5. Picks the closest one
 6. Extracts the full time series from that pixel
-

If you'd like, I can now show you:

- A more mathematically precise version
- A faster vectorized version
- Or a version using xarray's built-in masking instead of NumPy

Just tell me which direction you want to optimize.